

Assignment 1 — Git, NumPy & Pandas

Modules covered: 1. Version Control & Project Foundations · 2. NumPy Foundations · 3. Pandas DataFrame Basics & Selection

Instructor: Netra Pd. Neupane

Weight: 100 points (+8 bonus)

Estimated effort: 6–8 hours

Submission: one GitHub repository URL (see §6)

0. How this assignment works

Three parts, graded differently on purpose:

Part	Topic	Form	Points
A	Git & project foundations	Written answers. No repository history is graded — you explain the ideas in your own words.	30
B	NumPy	Code. Ten tasks on a fixed array dataset.	35
C	Pandas	Code. Twelve tasks on one deliberately messy CSV.	35

Two rules apply to every part:

1. **Answer in your own words, and show your output.** For Parts B and C, code that runs but prints nothing earns half marks at most. Every task must print a labelled result.
 2. **No loops where the library gives you a vectorized way.** Part B forbids `for` loops entirely except in one task that asks for one; Part C forbids `iterrows()`, `for i in range(len(df))`, and friends. That is the whole point of both libraries.
-

1. Setup

```
mkdir assignment-1 && cd assignment-1
python -m venv venv
# Windows: venv\Scripts\activate      macOS/Linux: source venv/bin/activate
pip install numpy pandas jupyter
pip freeze > requirements.txt
```

Copy the provided `make_data.py` into `data/`, then run it **once**:

```
cd data && python make_data.py      # writes scores_raw.csv
```

The seed inside is fixed, so everyone's `scores_raw.csv` is identical and every Part C task has exactly one correct answer. **Do not edit `make_data.py`, and do not hand-edit the CSV** — the cleaning is the assignment.

Sanity check before you start: the raw file has **126 rows and 7 columns**.

2. Part A — Git & project foundations (30 points)

Answer in `answers/01_git_theory.md`. **15 questions, 2 points each**. Two to five sentences per answer — enough to show you understand *why*, not just *what*. You may include a command or a snippet to illustrate a point, but the answer itself must be prose. Answers copied verbatim from the slides or generated by an AI tool score zero; write them as if explaining to a classmate who missed the session.

Git fundamentals

1. Git has three places a change can live: the working directory, the staging area, and the repository. Describe each, and explain what you would lose if the staging area did not exist.
2. `git init` and `git clone` both leave you with a Git repository. Explain what each one actually does, and give a situation where each is the right choice.
3. What does a commit store, and why is "committing" not the same as "saving a file"? Why is Git much less useful if `user.name` and `user.email` are unset or wrong?
4. `git status`, `git log`, and `git diff` answer three different questions. State the question each one answers, and describe a moment in your workflow where you would reach for each.
5. Explain what makes a commit message good. Why is "update" a genuine problem for a team six months later, and when is it worth writing a message *body* rather than just a summary line?

Remotes and the everyday workflow

6. Explain the relationship between your local repository and `origin`. What do `push` and `pull` each move, in which direction, and why is pulling before pushing the habit to build?
7. `git fetch` and `git pull` are not the same command. What is the difference, and when would you deliberately choose `fetch`?

Branching, merging, pull requests

8. A branch in Git is often described as "just a pointer." Explain what that means, and explain concretely what goes wrong on a team when everyone commits directly to `main`.
9. A merge conflict happens when two branches change the same lines of the same file. Explain why Git cannot resolve this automatically, what the `<<<<<<`, `====`, `>>>>>>` markers mean, and what you must do to finish the merge.
10. You could merge a branch locally with `git merge` and push. What does opening a **Pull Request** add that a local merge does not? What belongs in a PR description?

Issues

11. Explain the purpose of labels and assignees on an Issue, and what `Fixes #12` in a merged PR does. Why is linking work to Issues better than closing them by hand?

Project structure, environments, secrets

12. Why should `.gitignore` be one of your first commits? If a file is *already tracked*, does adding it to `.gitignore` stop Git from tracking it — and if not, what do you do instead?
 13. Explain the difference between `.env` and `.env.example`, and why they get opposite treatment. If a real API key was committed three weeks ago, why is deleting it in a new commit *not* a fix, and what should actually be done?
 14. What problem do a virtual environment and `requirements.txt` solve together? Why is `venv/` itself never committed, when `requirements.txt` always is?
 15. Explain what `git push --force` does to a shared branch and whose work it can destroy. How does `--force-with-lease` behave differently, and why is that safer?
-

3. Part B — NumPy (35 points)

Answer in `answers/02_numpy.py` (or `.ipynb`). Start the file with this exact block — it is your dataset for the whole part:

```
import numpy as np

np.random.seed(7)
cities = np.array(["Kathmandu", "Pokhara", "Biratnagar", "Lalitpur", "Birgunj"])
days = np.array([f"D{i:02d}" for i in range(1, 15)])

temps = np.random.randint(14, 36, size=(5, 14)).astype(float)    # °C, 5 cities × 14 days
humidity = np.random.randint(40, 95, size=(5, 14)).astype(float) # %

temps[2, 5] = np.nan      # sensor outage
temps[4, 11] = np.nan
```

No `for` loops, `while` loops, or list comprehensions anywhere in Part B except in task B5, which asks for one. Label every printed result, e.g. `print("B3a - readings above 30 per city:", ...)`.

B1 — Array anatomy and reshaping (3 pts)

Print `temps.shape`, `.ndim`, `.dtype`, `.size`. Then build `np.arange(70)`, reshape it to the same shape as `temps` two ways — with explicit dimensions and with `-1` — show the two results are identical, and flatten it back. In a comment, explain what `-1` asked NumPy to work out, and why reshaping does not copy the data.

B2 — Indexing and slicing (4 pts)

Using indexing and slicing only, print: (a) Pokhara's day-3 temperature; (b) every city's reading on the last day; (c) Biratnagar's full fortnight; (d) the block of the first three cities across days 4–8; (e) every other day for Lalitpur; (f) Kathmandu's fortnight reversed. Print the shape alongside each result.

B3 — Boolean masks (3 pts)

(a) Build the mask `temps > 30` and print how many readings exceed 30 **per city** — one number per city, no loop. (b) Print the actual day labels on which Kathmandu exceeded 30, by using the mask to index `days`. (c)

In a comment, explain in one sentence what a mask *is*, and why `temps[temps > 30]` cannot keep the 2D shape.

B4 — Views vs. copies (3 pts)

Take a slice of one city's first three days, set its first element to `999`, and print the original array to prove it changed. Repeat with `.copy()` and prove it did not. In a comment, explain in two sentences why NumPy defaults to a view, and what bug this causes if you forget.

B5 — Vectorization vs. a Python loop (4 pts)

Convert `temps` to Fahrenheit two ways: with an explicit loop (the only loop allowed in Part B) and with one vectorized expression. Confirm they agree with `np.allclose`. Then time both on an array of 1,000,000 values using `time.perf_counter()`, and print both timings plus the speedup as a multiple (`~Nx faster`). In a comment, explain where the loop's extra time actually goes.

B6 — Broadcasting (4 pts)

(a) Compute each city's own fortnight average with a NaN-aware function and `keepdims=True`, then subtract it from `temps` to get an anomaly grid; print the result's shape and confirm it is still `(5, 14)`. (b) Apply a per-day sensor correction of shape `(14,)` to all five cities in one expression. (c) Attempt `temps + np.array([1, 2, 3, 4, 5])`, catch the `ValueError`, print its message, and explain in a comment why aligning shapes *from the right* makes this fail while (b) succeeds.

B7 — Matrix multiplication (4 pts)

(a) On the grocery-basket example (`quantities = [2, 3, 1]`, `prices = [120, 90, 250]`), print both `*` and `@`, and state in a comment what each returned and why they differ. (b) Build `features` of shape `(14, 2)` whose columns are the across-city NaN-aware mean temperature and mean humidity per day, define `weights = np.array([0.6, -0.1])`, and compute a comfort score per day with `@`. Print the shapes of `features`, `weights`, and the result, and explain in a comment why the inner dimensions had to match.

B8 — Aggregating along an axis (4 pts)

Using NaN-aware aggregations, print: mean, max, min and std **per city**; mean and max **per day**; and each city's hottest day as a *day label* (use `argmax`, then index `days`). State each result's shape. In a comment, state what `axis=0` and `axis=1` collapse here, in terms of this dataset.

B9 — Reproducible randomness and a train/test split (3 pts)

Flatten `temps` to 70 samples. With `np.random.seed(42)`, shuffle an index array and split it 80/20 into train and test index sets; print both shapes, confirm they sum to 70, and confirm they share no index. Re-seed with 42, repeat, and assert the split is identical. In a comment, explain why you shuffle *before* splitting, and why the seed matters when you share results.

B10 — `np.where`, `argmax`, `argmin` (3 pts)

(a) Label every reading "hot" / "mild" / "cold" (above 30, 22–30, below 22) with nested `np.where`, and print the label grid's shape. (b) Given `scores = np.array([0.08, 0.62, 0.21, 0.09])` and `labels = ["rain", "clear", "cloudy", "fog"]`, print the predicted label using `argmax`. In a comment, explain why a classifier needs `argmax` rather than `max`.

4. Part C — Pandas (35 points)

Answer in `answers/03_pandas.ipynb`, using `data/scores_raw.csv`. **No row loops, no `iterrows()`, and no `apply()` where a vectorized expression or a built-in method exists.**

The raw file has seven columns — `student_id`, `name`, `city`, `subject`, `marks`, `attendance_percent`, `exam_date` — and it is dirty on purpose: inconsistent city spellings, blank values, exact duplicate rows, marks that are not physically possible, and dates stored as text.

C1 — Series and DataFrame by hand (2 pts)

Build a `Series` of five marks with student names as the index, and print its `.index` and `.values`. Build the same small table as a `DataFrame` twice — once from a dictionary of lists, once from a list of dictionaries — and show the two are equal. Print `.shape`, `.columns`, `.index`.

C2 — Load and inspect (3 pts)

Read `scores_raw.csv`. Print `head()`, `tail(3)`, `shape`, `info()`, `describe()`, and `describe(include="object")`. Then write two to three sentences naming **three specific problems** visible in that output, each backed by a number or a value from it.

C3 — Selecting rows and columns (3 pts)

Print `df["marks"]` and `df[["marks"]]` with `type()` of each, and say in a comment what the difference is. Then show `df.loc[0:4, ["name", "subject", "marks"]]` and `df.iloc[0:5, 1:4]`, print how many rows each returned, and explain in a comment why the two slices treat their end point differently.

C4 — Boolean filtering (3 pts)

(a) Rows with `marks > 80` **and** subject `"Python Programming"`. (b) Rows from Kathmandu **or** Pokhara — note that this under-counts until you clean the city column in C5, and say so. (c) The same filter as (a) written with `.query()`, proved identical with `.equals()`. In a comment, explain why `and` / `or` fail inside a bracket filter but are required inside `query()`.

C5 — Cleaning inconsistent text (3 pts)

Print `df["city"].nunique()` and `value_counts()` before cleaning. Normalise the column with string methods (strip whitespace, fix capitalisation), then print both again. **Checkpoint: 24 distinct spellings before, 6 after.** Re-run your C4(b) filter and report how many rows you were losing.

C6 — Missing values (3 pts)

Print `isna().sum()` per column. Fill missing `attendance_percent` with the column mean; drop the rows with missing `marks`. Print the row count before and after each step. Then, in two sentences, argue whether mean-filling attendance was a defensible choice here.

C7 — Duplicates (3 pts)

Using `duplicated()`, print how many fully duplicated rows exist and show a few. Drop them keeping the first occurrence, and print the shape before and after. **Checkpoint: 6 duplicates, 126 → 120 rows.** Separately, report duplicates on the subset `["student_id", "subject"]`, and explain in a comment why that subset is the one that actually matters here.

C8 — Dates (3 pts)

Print `exam_date`'s dtype, convert it with `pd.to_datetime()`, and print the dtype again. Add `exam_month` and `exam_weekday` columns using the `.dt` accessor, then filter to exams held after `2026-02-15` and print how many there are.

C9 — Sorting and renaming (3 pts)

Sort by `subject` (A–Z) and, within each subject, `marks` (high to low); show the top rows. Rename `attendance_percent` to `attendance` **without modifying the original DataFrame**, and prove the original is unchanged by printing both `.columns`. Then restore the original row order with `sort_index()`.

C10 — Derived columns (3 pts)

Add `passed` (`marks >= 40`), and `grade` using nested `np.where`: 85+ → A, 70–84 → B, 40–69 → C, below 40 → F. Print the overall pass rate as a percentage by taking the mean of `passed` cast to `int`, and print `grade.value_counts()`. Deal with the impossible marks values you found in C2, and say in a comment what you did with them and why.

C11 — Types and memory (3 pts)

Convert `subject` and `city` to the `category` dtype. Print `memory_usage(deep=True)` for those two columns before and after, in bytes, and the percentage saved. In a comment, explain what makes `category` cheaper, and when it would *not* help.

C12 — Aggregation and a summary table (3 pts)

On the cleaned data, produce: (a) mean, max and count of `marks` per subject using `groupby().agg()`; (b) mean `marks` by `city` and `subject`; (c) a `pivot_table` of mean marks with cities as rows and subjects as columns; (d) mean marks per `exam_month`, sorted chronologically. Export (a) to `outputs/summary.csv`, and state in one sentence what the month trend shows.

5. Bonus (up to +8)

- **+5 — A reporting pipeline.** Wrap Part C into three functions — `load(path)`, `clean(df)`, `summarise(df)` — so that `summarise(clean(load("data/scores_raw.csv")))` reproduces your C12(a) table from the raw file in one line. It must run top to bottom with no manual steps.
 - **+3 — Where your cleaning could mislead.** 150–250 words in `answers/03_pandas.ipynb`: pick two cleaning decisions you made (mean-filling attendance, dropping missing marks, deleting duplicates, whatever you did with the impossible marks) and explain, concretely, how each one could bend the conclusions in your C12 summary.
-

6. Deliverables and submission

Please push this assignment in your github repository in following format: the URL of your GitHub repository, laid out like this:

```

assignment-1/
├── answers/
│   ├── 01_git_theory.md
│   ├── 02_numpy.py          (or 02_numpy.ipynb)
│   └── 03_pandas.ipynb
├── data/
│   ├── make_data.py
│   └── scores_raw.csv
├── outputs/
│   └── summary.csv
├── .gitignore
├── requirements.txt
└── README.md

```

- `README.md` : what the project is, how to create the venv, install dependencies, generate the data, and run each part. Assume the reader has never seen it.
- Notebooks must be committed **with their outputs saved**. A notebook whose cells show no output cannot be graded as code that ran.
- `venv/` and `__pycache__/` must not be in the repository — that is what `.gitignore` is for, and Part A question 12 is about exactly this.
- The repository must be public, or private with the instructor added as a collaborator.

Deliverable: Please upload your answers in leading2ai platform in pdf. The pdf should contain Questions as well as answer

Self-check before submitting:

```

python data/make_data.py          # regenerates the identical CSV
python answers/02_numpy.py        # runs top to bottom, no errors
jupyter nbconvert --execute --to notebook --inplace answers/03_pandas.ipynb
git status                        # clean; no venv/, no __pycache__

```

7. Grading rubric — 100 points

Part	Area	Points	What earns full marks
A	Git & foundations (15 × 2)	30	Each answer correct, in your own words, and explaining <i>why</i> — not a restated definition. Partial credit for a correct but shallow answer.
B	NumPy B1–B10	35	Correct result, correct tool, labelled output, no loops. A right answer reached with a loop scores half.
C	Pandas C1–C12	35	Correct result, correct method, output visible, checkpoints matched, and the short written judgements actually engage with the data.

8. Where each session section is assessed

Session section	Assessed by
1 · 01 Git fundamentals	A1–A5
1 · 02 GitHub & repositories	A6, A7
1 · 03 Core workflow	A4, A6, A7
1 · 04 Branching & Pull Requests	A8–A10
1 · 05 Issues for collaboration	A11
1 · 06 Structure & environments	A12–A15, §6 layout
2 · Arrays, indexing, slicing	B1–B4
2 · Vectorized operations, broadcasting	B5, B6
2 · Matrix multiplication, axis aggregation	B7, B8
2 · Random numbers, <code>where</code> / <code>argmax</code> , cleanup	B9, B10
3 · Series & DataFrames, loading & inspecting	C1, C2
3 · Selection, <code>loc</code> / <code>iloc</code> , boolean filtering, <code>query</code>	C3, C4
3 · Sorting, renaming, new columns	C9, C10
3 · Types & dates, cleanup	C5–C8, C11
3 · Aggregation & reporting	C12, bonus